

“Manual de Sistemas Operativos 1”

Jairo Aguilera

Universidad Nacional Autónoma de Honduras del Valle de Sula

SO1: Sistemas Operativos 1

Norman Cubilla

19 de abril del 2026

Automatización de procesos dentro del SO Linux

Existe un tipo de archivo de extensión tipo sh que permite la creación de proceso dentro del sistema operativo Linux. Para la automatización de dichos procesos se debe crear el archivo mediante el comando [pico].

Este comando crea este tipo de archivo mediante el cual se pueden automatizar los procesos dentro de este entorno de pico nos permite dar instrucciones mediante palabras restringidas para la realización de acciones específicas como ser: echo (para imprimir en el la consola), read (la leer mediante el ingreso de cadenas de texto), y touch para la creación de directorios. Todo este proceso se debe realizar con los permisos de orden superior "sudo su".

Antes de cualquier comando de dicho archivo se debe agregar el #!/bin/bash para que el pequeño programa funcione bien.

```
GNU nano 6.2
#!/bin/bash
cd /home/username/Desktop/Lab01
clear
echo "Creacion de Archivos"
echo ""
read -p "Ingrese la el nombre del directorio: " direc
read -p "Ingrese el nombre del archivo: " file
touch $direc/$file
sleep 3
tree /home/username/Desktop
```

Justamente para que el archivo se muestre en el directorio y que el archivo tenga todo tipo de permiso para poder realizar las acciones establecidas se le debe dar un tipo de formato aplicando el siguiente comando:

`--chmod 777 NombreDelArchivo.sh`

“Esto sirve para darle permisos de (todo abierto) al archivo prácticamente es como darle un sudo su al archivo.”

Una vez se tiene hecho el archivo.sh lo siguiente es probarlo, para correr el archivo se debe aplicar el siguiente código:

`--./archivo.sh`

Ejemplo del programa en ejecución:

```
Creacion de Archivos
Ingrese la el nombre del directorio: dir08
Ingrese el nombre del archivo: file08
/home/username/Desktop
├── Lab01
│   ├── createfile.sh
│   ├── dir01
│   ├── dir02
│   │   └── file01
│   ├── dir03
│   ├── dir04
│   ├── dir05
│   ├── dir06
│   ├── dir07
│   ├── dir08
│   │   └── file08
│   ├── mnt
│   │   └── disco2_app1
│   └── more
│       └── movefile.sh
11 directories, 5 files
root@server:/home/username/Desktop/Lab01#
```

“Aquí se logra ver como hemos creado un archivo en el directorio dir08 llamado file 08”.

También se puede utilizar la creación de programas con otro fin he aquí otro ejemplo:

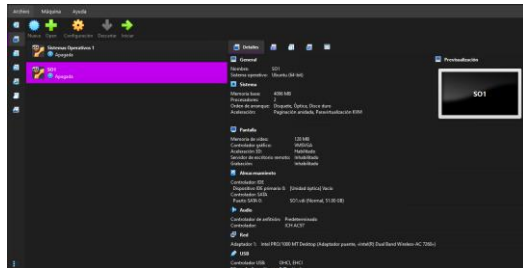
```
root@server:/home/username/Desktop/Lab01# ./movefile.sh
Moviendo archivos
Ingrese el nombre del archivo: dir08/file08
Ingrese la ruta de destino del archivo: dir05
/home/username/Desktop/Lab01
├── createfile.sh
├── dir01
├── dir02
│   └── file01
├── dir03
├── dir04
├── dir05
│   └── file08
├── dir06
├── dir07
├── dir08
├── mnt
│   └── disco2_app1
├── more
└── movefile.sh
10 directories, 5 files
root@server:/home/username/Desktop/Lab01#
```

“Aquí podemos ver como creamos un programa con el fin de mover archivos entre directorios, hemos movido el file08 del dir08 al dir05”.

(El tree es algo que se debe instalar con el código `--sudo apt-get install tree`).

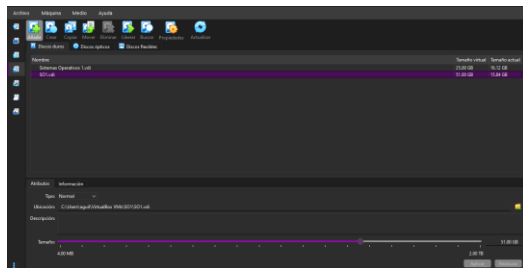
Expandir memoria del Virtual Box

Para expandir la memoria del virtual box lo que tenemos que hacer es primeramente apagar la maquina donde tengamos nuestro sistema operativo. Luego de eso seleccionar la maquina donde vamos a expandir la memoria.

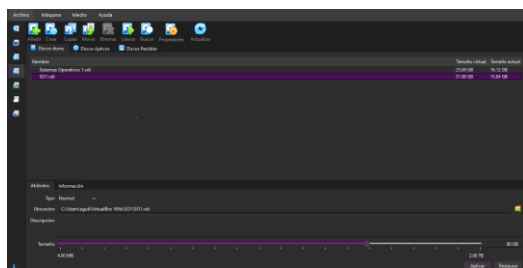


“sinos fijamos actualmente nuestra maquina cuenta con un total de 51 gb nosotros lo vamos a aumentar a 60”

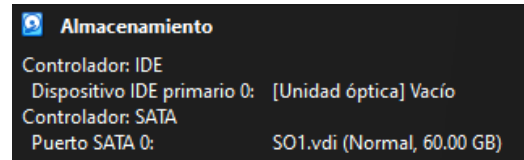
Para aumentar el almacenamiento vamos a irnos a la sección de medios seleccionando nuestra maquina



Una vez la tengamos seleccionada vamos a aumentarle la barra de abajo a 60gb y ya con eso nuestra maquina debería tener el aumento de 10gb que le hicimos mediante el virtual box.



Le daremos en aplicar y ya nos debería aparecer el almacenamiento que le dimos.



Creación de particiones en el SO Linux:

Para la creación de particiones en Linux primero debemos verificar las particiones ya establecidas por el sistema ejecutando el comando “lsblk” el cual permite verificar como esta administrado el sistema operativo.

```
root@sistemas1:/home/sistemas1# lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
loop0       7:0      0   4K  1 loop /snap/bare/5
loop1       7:1      0  63.9M 1 loop /snap/core20/2318
loop2       7:2      0  66.8M 1 loop /snap/core24/1587
loop3       7:3      0  273.5M 1 loop /snap/firefox/8054
loop4       7:4      0  606.1M 1 loop /snap/gnome-46-2404/153
loop5       7:5      0  91.7M 1 loop /snap/gtk-common-themes/1535
loop6       7:6      0   87M 1 loop /snap/lxd/29351
loop7       7:7      0  91.7M 1 loop /snap/lxd/38800
loop8       7:8      0  38.8M 1 loop /snap/snapd/21759
loop9       7:9      0  395M 1 loop /snap/mesa-2404/1165
sda         8:0      0   51G  0 disk
├─sda1      8:1      0    1M  0 part
├─sda2      8:2      0   25G  0 part /
└─sr0       11:0     1  50.7M  0 rom  /media/sistemas1/VBox_GAs_7.2.4
root@sistemas1:/home/sistemas1#
```

“En pocas palabras desglosa como está dividido el almacenamiento del SO de Linux”.

Para crear una partición debemos ingresar el siguiente comando:

```
--sudo fdisk /dev/sda
```

Ingresando el código en consola aparecerá un pequeño apartado, en la misma consola donde se deberá colocar el nombre de la partición, el tamaño y otros datos que se piden para la creación de la misma.

```
root@sistemas1:/home/sistemas1# sudo fdisk /dev/sda
welcome to fdisk (util-linux 2.37.2).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

GPT PMBR size mismatch (52428799 != 106947071) will be corrected by write.
The backup GPT table is not on the end of the device. This problem will be corrected by write.
This disk is currently in use - repartitioning is probably a bad idea.
It's recommended to unmount all file systems, and swapoff all swap
partitions on this disk.

Command (n for help):
```

“Al momento de que aparezca el siguiente apartado debemos ingresar la letra *n* para crear una nueva partición, y luego debemos ingresar el numero de la partición”

```
Command (n for help): n
Partition number (3-128, default 3): 3
First sector (52426752-106947038, default 52426752):
Last sector, +/-sectors or +/-size(K,M,G,T,P) (52426752-106947038, default 106947038):
```

Por último, ingresamos el tamaño de la partición ingresando en pantalla un + para aumentar su capacidad, y luego ingresando el número y la letra del tipo de dato en este caso GByte.

```
Command (m for help): n
Partition number (3-128, default 3): 3
First sector (52426752-106947038, default 52426752):
Last sector, +/-sectors or +/-size[K,M,G,T,P] (52426752-106947038, default 106947038): +10G

Created a new partition 3 of type 'Linux filesystem' and of size 10 GiB.
Command (m for help):
```

Una vez listo ingresamos la letra w y así nos salimos del pequeño apartado de las particiones. Por último, corroboramos que la partición se halla creado ingresando en el comando *lsblk*.

```
Command (m for help): w
The partition table has been altered.
Syncing disks.

root@sistemas1:/home/sistemas1# lsblk
NAME        MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
loop0       7:0      0    4K  1 loop /snap/bare/5
loop1       7:1      0   63.9M  1 loop /snap/core20/2318
loop2       7:2      0   66.8M  1 loop /snap/core24/1587
loop3       7:3      0   273.5M  1 loop /snap/firefox/8054
loop4       7:4      0   606.1M  1 loop /snap/gnome-46-2404/153
loop5       7:5      0   91.7M  1 loop /snap/gtk-common-themes/1535
loop6       7:6      0    87M  1 loop /snap/lxd/29351
loop7       7:7      0   91.7M  1 loop /snap/lxd/38800
loop8       7:8      0   38.8M  1 loop /snap/snapd/21759
loop9       7:9      0   395M  1 loop /snap/ mesa-2404/1165
sda         8:0      0    51G  0 disk
├─sda1      8:1      0    1M  0 part
├─sda2      8:2      0   25G  0 part /
├─sda3      8:3      0   10G  0 part
└─sr0       11:0     1   50.7M  0 rom  /media/sistemas1/VBox_GAs_7.2.4
root@sistemas1:/home/sistemas1#
```

("Importante se debe corroborar que la maquina virtual en sus configuraciones tenga un suficiente espacio, porque si no lo tiene las particiones que se crearan serán de un tamaño muy pequeño")

Lo último sería reiniciar el SO Linux para que se reconozca el sda3 para darle formato y poder montar la partición.

--sudo reboot

Bien ahora que tenemos hecha la partición debemos darle el formato ext4 para así crear un punto de acceso y poder montar esa partición en un directorio, para eso utilizamos el siguiente comando

--sudo mkfs.ext4 /dev/NombrePartición

```
root@sistemas1:/home/sistemas1# sudo mkfs.ext4 /dev/sda3
mkfs.ext4 1.46.5 (30-Dec-2021)
Creating filesystem with 2621440 4k blocks and 655360 inodes
Filesystem UUID: df99a2f0-65a2-4378-a301-db649abdb03
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done

root@sistemas1:/home/sistemas1#
```

Montar particiones previamente creadas en el SO de Linux.

Para montar particiones en el SO Linux primeramente debemos crear un punto de acceso, es decir un directorio donde estará ubicado el desglose de la partición. Para eso debemos crear un directorio con el nombre de *mnt*.

--mkdir mnt/sda3

"Esto sería un ejemplo, el directorio lo podemos crear en cualquiera ubicación del sistema operativo. En mi caso creare otro usuario con un directorio de nombre *username/Desktop/mnt* esto para no mezclarlo con el usuario original y encontrar la partición más rápido".

```
root@sistemas1:/home# ls -l
total 4
drwxr-x--- 17 sistemas1 sistemas1 4096 Apr  8 17:30 sistemas1
root@sistemas1:/home# mkdir username
root@sistemas1:/home# cd username
root@sistemas1:/home/username# mkdir Desktop
root@sistemas1:/home/username# mkdir Desktop/mnt
```

"y dentro de mnt creamos el directorio donde estará ubicada la partición."

```
root@sistemas1:/home/username# mkdir Desktop/mnt/sda3
```

Bien ahora que tenemos listo eso, llego el momento de montar la partición sda3 que creamos anteriormente. Para eso debemos ingresar a comando la siguiente línea de código.

--sudo mount /dev/sda3 Desktop/mnt/sda3

```
root@sistemas1:/home/username# sudo mount /dev/sda3 Desktop/mnt/sda3
```

Ahora que lo tenemos listo, solo debemos verificar que la partición se halla montado en esa ubicación para eso ingresamos *df -h* y debería aparecernos el nombre de la

partición que montamos, y también la ubicación de la partición montada.

```
root@sistemas1:/home/username# df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs            392M  1.5M  390M   1% /run
/dev/sda2        25G   9.8G   14G  43% /
tmpfs            2.0G   0  2.0G   0% /dev/shm
tmpfs            5.0M  4.0K  5.0M   1% /run/lock
tmpfs            392M  116K  392M   1% /run/user/1000
/dev/sr0         51M   51M   0 100% /media/sistemas1/VBox_GAs_7.2.4
/dev/sda3        9.8G   24K   9.3G   1% /home/username/Desktop/mnt/sda3
root@sistemas1:/home/username#
```

Y así hemos creado una partición, a esta también se le pueden dar permisos, así como a los archivos.sh que creamos anteriormente, el comando será el mismo:

```
--sudo chmod 777 Destop/mnt/sda3
```

Bien ahora viene lo más importante que es ingresar la partición al archivo fstab para así cada vez que reiniciemos el SO o lo apaguemos y los volvamos a encender, no tengamos que estar montando la partición a cada rato. Para eso ingresamos el comando *blkid*.

```
root@sistemas1:/home/sistemas1# blkid
/dev/sda2: UUID="977d9482-9ba4-4dbe-92bd-82955994575d" BLOCK_SIZE="4096" TYPE="ext4" PARTUUID="c1b7f1fe-d9bc-4513-8786-8b95f5943a60"
/dev/sda3: UUID="df99a2f0-65a2-4378-a3b1-d049abdb0b03" BLOCK_SIZE="4096" TYPE="ext4" PARTUUID="d33a0b03-6eba-3b45-ad00-f457ee365455"
/dev/loop1: TYPE="squashfs"
/dev/loop6: TYPE="squashfs"
/dev/loop4: TYPE="squashfs"
/dev/loop2: TYPE="squashfs"
/dev/loop8: TYPE="squashfs"
/dev/loop9: TYPE="squashfs"
/dev/loop7: TYPE="squashfs"
/dev/sda1: PARTUUID="837beacb-a30b-4c73-b0b6-39385b57d9d7"
/dev/loop5: TYPE="squashfs"
/dev/loop3: TYPE="squashfs"
root@sistemas1:/home/sistemas1#
```

“Y lo que nos interesa es el código UUID de la partición que vamos a montar”

Lo que debemos hacer es copiar ese UUID y pegarlo en un notpad, luego entramos a los directorios raíz etc.

```
--cd /etc
```

Nuestro objetivo es estando dentro de etc encontrar el archivo fstab ingresando un:

```
--ls -l fstab
```

```
-rw-r--r-- 1 root root 473 Apr  8 04:40 fstab
root@sistemas1:/etc#
```

Luego editamos el archivo con un pico:

```
--pico fstab
```

Y luego ingresamos justo debajo de donde está la partición primaria lo siguiente:

```
--UUID=eINumeroqueCopiamosdeSda3
ubicaciónParticion ext4 defaults 0 2
```

```
GNU nano 6.2          fstab
#
# /etc/fstab: static file system information.
#
# Use 'blkid' to print the universally unique identifier for a
# device; this may be used with UUID= as a more robust way to name devices
# that works even if disks are added and removed. See fstab(5).
#
# file system=mount point= <type> <options> <dump> <pass>
# / was on /dev/sda2 during curtin installation
/dev/disk/by-uuid/977d9482-9ba4-4dbe-92bd-82955994575d / ext4 defaults 0 1
/swap.img none swap sw 0 0
# particion de la particion sda3
UUID=df99a2f0-65a2-4378-a3b1-d049abdb0b03 /home/username/Desktop/mnt/sda3 ext4 defaults 0 2
```

Por último, salimos tecleando un ctrl+o guardando, y luego con un ctrl+x salimos. Para verificar todo, ingresamos el siguiente código:

```
--sudo mount -a
```

Si todo sale bien, no debería lanzar ni un solo error, por último, reiniciamos y ya la partición debería estar totalmente lista para su uso.

```
--sudo reboot
```

```
root@sistemas1:/home/sistemas1# df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs            392M  1.5M  390M   1% /run
/dev/sda2        25G   9.8G   14G  43% /
tmpfs            2.0G   0  2.0G   0% /dev/shm
tmpfs            5.0M  4.0K  5.0M   1% /run/lock
/dev/sda3        9.8G   24K   9.3G   1% /home/username/Desktop/mnt/sda3
tmpfs            392M  112K  392M   1% /run/user/1000
```

“Para verificar que la partición ya está montada ejecutamos el comando *df -h*, y ya debería salir nuestra partición montada y creada”.

```
sistemas1@sistemas1:/lib/systemd/system$ cd /var/www/html
sistemas1@sistemas1:/var/www/html$ ls -l
total 12
-rw-r--r-- 1 root root 10671 Apr  9 15:36 index.html
sistemas1@sistemas1:/var/www/html$
```

Instalación y desarrollo de Apache

Primero debemos instalar las utilidades de apache lo primero que debemos hacer es ingresar el siguiente código a la terminal para así poder hacer la instalación del apache:

```
--sudo apt-get install apache2 apache2-utils
```

Una vez haya finalizado comenzaremos a verificar si está corriendo para eso ingresaremos el siguiente comando:

```
--/etc/init.d/apache2 status
```

“Este comando solo es para verificar que lo que acabamos de instalar este corriendo”.

```
sistemas1@sistemas1:~$ /etc/init.d/apache2 status
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/lib/systemd/system/apache2.service; enabled; vendor preset: enabled)
   Active: active (running) since Thu 2026-04-09 15:36:52 UTC; 1min 52s ago
     Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 2779 (apache2)
      Tasks: 55 (limit: 4547)
     Memory: 5.0M
        CPU: 139ms
    CGroup: /system.slice/apache2.service
            └─2779 /usr/sbin/apache2 -k start
              └─2780 /usr/sbin/apache2 -k start
                └─2781 /usr/sbin/apache2 -k start

Apr 09 15:36:52 sistemas1 systemd[1]: Starting The Apache HTTP Server...
Apr 09 15:36:52 sistemas1 apache2ctl[2773]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.0.1 instead; this will likely move into the future using a more meaningful FQDN.
Apr 09 15:36:52 sistemas1 systemd[1]: Started The Apache HTTP Server.
Hint: Some lines were ellipsized, use -l to show in full.
```

Nuestro apache está alojada en la siguiente ruta de directorio a la cual si deseamos ingresar ser ingresar de la siguiente manera:

```
--cd /lib/systemd/system
```

Y luego buscar el archivo de apache ingresando una `ls -l apache2` y así nos deberían aparecer los archivos de apache:

```
sistemas1@sistemas1:~$ cd /lib/systemd/system
sistemas1@sistemas1:/lib/systemd/system$ ls -l apache2*
-rw-r--r-- 1 root root 419 Mar 18 2024 apache2.service
-rw-r--r-- 1 root root 491 Mar 18 2024 apache2@.service
sistemas1@sistemas1:/lib/systemd/system$
```

Luego podemos ingresar a otra ruta de directorio para poder ver el archivo que genera la instalación de apache siendo una `index.html` con la siguiente ruta, y también haciendo un `ls -l` dentro de ella:

```
--cd /var/www/html
```

Luego podemos ingresar a la ip de la SO de Linux utilizando nuestra ip de la maquina e ingresándola en nuestro navegador y así obtener una pantalla un poco peculiar. Esto lo logramos obteniendo la ip de nuestra maquina con el comando `ip a.` y una vez obtenida podemos ingresar dicha ip a nuestro navegador y así lograr ver la pantalla que el mismo navegador nos lanza.

```
sistemas1@sistemas1:/var/www/html$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:47:9d:91 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.84/24 metric 100 brd 192.168.1.255 scope global dynamic enp0s3
        valid_lft 84546sec preferred_lft 84546sec
    inet6 fe80::a00:27ff:fe47:9d91/64 scope link
        valid_lft forever preferred_lft forever
sistemas1@sistemas1:/var/www/html$
```

“Nuestra ip en este caso es 192.168.1.84 esta es la que debemos ingresar en nuestro navegador”.



Esta sería la página que nos debería salir esto es la página que trae por default el apache que instalamos en nuestro sistema operativo.

Dominio creado en la partición sda3 que creamos anteriormente

Bien ahora ya habiendo establecido la instalación del apache ya corroborando que todo anda bien, ahora nuestro objetivo será crear un dominio local en el punto de la partición `sda3` que creamos con anterioridad. Para eso lo primero será

ingresar a nuestra particion y crear un directorio llamado *public_app1*.

--mkdir public_app1.

```
root@sistemas1:/home/sistemas1# cd /home/username/Desktop/mnt/sda3
root@sistemas1:/home/username/Desktop/mnt/sda3# mkdir public_app1
root@sistemas1:/home/username/Desktop/mnt/sda3# ls -l
total 20
drwx----- 2 root root 16384 Apr  8 22:25 lost+found
drwxr-xr-x 2 root root 4096 Apr  9 18:44 public_app1
root@sistemas1:/home/username/Desktop/mnt/sda3#
```

Una vez lo tenemos hecho nuestro siguiente paso debe ser crear un archivo *index.html* dentro del directorio *public_app1* para eso usaremos el comando *pico*.

--pico index.html

```
root@sistemas1:/home/username/Desktop/mnt/sda3/public_app1# pico index.html
```

Nuestro archivo aparecerá en blanco, nosotros llenaremos el archivo de la siguiente manera utilizando el lenguaje de programación de html que se basa en etiquetas.

```
GNU nano 6.2 index.html
<html>
  <head>
    <h1>Proyecto creado en nuestro sda3</h1>
  </head>
  <body>
    <h1> Proyecto Web de nuestro sda3 </h1>
  </body>
</html>
```

“No necesariamente debe quedar igual a este solo es por motivos de ejemplo”

Ahora que lo tenemos listo guardamos con *ctrl+o* y luego cerramos con un *ctrl+x*. Listo eso, nuestro siguiente objetivo es ingresar al siguiente directorio de nuestra partición base:

--cd /etc/apache2/sites-available/

Dentro de este directorio crearemos un archivo de extensión tipo conf esto para crear el servidor local y poder correr nuestro correrlo.

```
root@sistemas1:/home/username/Desktop/mnt/sda3/public_app1# cd /etc/apache2/sites-available
root@sistemas1:/etc/apache2/sites-available# ls -l
total 12
-rw-r--r-- 1 root root 1332 Mar 18 2024 000-default.conf
-rw-r--r-- 1 root root 6338 Mar 18 2024 default-ssl.conf
root@sistemas1:/etc/apache2/sites-available# pico app1.conf
```

Dentro del archivo.conf ingresaremos lo siguiente siempre con la utilización de etiquetas.

```
GNU nano 6.2 app1.conf
<VirtualHost *:80>
  ServerName www.app1.local
  DocumentRoot /home/username/Desktop/mnt/sda3/public_app1
  <Directory /home/username/Desktop/mnt/sda3/public_app1>
    Options Indexes FollowSymLinks
    AllowOverride All
    Require all granted
  </Directory>
</VirtualHost>
```

“Abriremos el virtual host y en *ServerName* ingresaremos el nombre de nuestro dominio local con un *www/nombre.local*. Luego en *DocumentRoot* ingresaremos la ubicación del archivo *index.html*. Luego abriremos una etiqueta llamada *Directory* siempre con la ubicación del archivo *index.html* y le daremos los siguientes permisos, esto para que se pueda ver de manera default tal como aparece en la imagen. Por último, cerraremos la etiqueta *Directory* y también la virtual host. Listo eso guardamos y nos salimos del archivo”.

Ahora que tenemos esto hecho pasamos a uno de los últimos pasos que sería ingresar al directorio siguiente:

--cd /etc

Dentro de este directorio ingresaremos al archivo *hosts* esto siempre con el comando *pico*.

--pico hosts

```
GNU nano 6.2 hosts *
127.0.0.1 localhost
127.0.1.1 sistemas1

# The following lines are desirable for IPv6 capable hosts
::1 ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
```

Nos debería aparecer lo de la imagen anterior. Justo debajo de todo ingresaremos el siguiente código:

--127.0.0.1 ServerName

“Hay que recordar que en este caso nuestro *ServerName* es *www.app1.local*”.

--ipdelamaquina *www.app1.local*

“El ip de la maquina se obtiene con el comando *ip a*”.

```
GNU nano 6.2 hosts *
127.0.0.1 localhost
127.0.1.1 sistemas1

# The following lines are desirable for IPv6 capable hosts
::1 ip6-localhost ip6-loopback
fe80::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters

# Dominio del sda3 de mi servidor local
127.0.0.1 www.app1.local
192.168.1.84 www.app1.local
```

Nos debería de quedar de la siguiente manera. Una vez lo tengamos guardamos y nos salimos del archivo.

Ahora que tenemos listo, nuestro siguiente paso debería ser activar nuestro dominio y servidor para eso debemos irnos a la siguiente ruta de directorio:

--*cd /etc/apache2/sites-enabled*

Dentro de este directorio tenemos los archivos que están activados y corriendo en nuestro servidor local.

```
root@sistemas1:/etc/apache2/sites-enabled# cd /etc/apache2/sites-enabled
root@sistemas1:/etc/apache2/sites-enabled# ls -l
total 0
lrwxrwxrwx 1 root root 35 Apr 9 15:36 000-default.conf -> ../sites-available/000-default.conf
root@sistemas1:/etc/apache2/sites-enabled#
```

Aquí nosotros debemos activar el servidor local que creamos anteriormente con el comando siguiente:

--*sudo a2ensite archivo.conf*

“Recuerda que para nuestro caso el archivo.conf era *app1.conf*”.

```
root@sistemas1:/etc/apache2/sites-enabled# sudo a2ensite app1.conf
Enabling site app1.
To activate the new configuration, you need to run:
systemctl reload apache2
root@sistemas1:/etc/apache2/sites-enabled#
```

Por último, recargamos nuestro apache con el siguiente código y ya nuestro dominio debería estar listo para poder correrlo:

--*systemctl reload apache2*

```
root@sistemas1:/etc/apache2/sites-enabled# systemctl reload apache2
root@sistemas1:/etc/apache2/sites-enabled# /etc/init.d/apache2 restart
Restarting apache2 (via systemctl): apache2.service.
root@sistemas1:/etc/apache2/sites-enabled#
```

“El código [*/etc/init.d/apache2 restart*] lo ingresamos para verificar que no haya ningún tipo de problema con nuestro dominio”.

Por último, dentro de nuestra Linux ingresamos a un navegador e introducimos al servername que le colocamos a nuestro dominio en este caso: *www.app1.local* y ya debería aparecernos en la página lo que nosotros ingresamos en un principio en el index.html de nuestra partición *sda3*.



Proyecto creado en nuestro sda3
Proyecto Web de nuestro sda3

SQL en el Sistema Operativo Linux

Antes de todo lo primero debe ser verificar la update de SO Linux que este todo en orden para eso el siguiente comando:

--*sudo apt update*

Lo primero es instalar el MySQL es nuestro SO de Linux para eso tenemos que poner en nuestra consola el siguiente comando:

--*sudo apt-get install mysql-server*

“Para evitar errores estaría bien si reiniciamos nuestro SO con una *sudo reboot*”

Una vez listo lo siguiente antes de continuar es verificar si nuestro MySQL server está corriendo, aplicando el siguiente comando:

--*/etc/init.d/mysql status*

```
root@sistemas1:~# /etc/init.d/mysql status
mysql.service - MySQL Community Server
Loaded: loaded (/lib/systemd/system/mysql.service; enabled; vendor preset: enabled)
Active: active (running) since Thu 2020-04-09 23:10:14 UTC; 22min ago
Process: 800 ExecStartPre=usr/share/mysql/mysql-systemd-start pre (code=exited, status=0/SUCCESS)
Main PID: 908 (mysqld)
Status: "Server is operational"
Tasks: 37 (limit: 4547)
Memory: 432.7M
CPU: 20.21%
CGroup: /system.slice/mysql.service
        └─908 /usr/sbin/mysqld

Apr 09 23:10:05 sistemas1 systemd[1]: Starting MySQL Community Server...
Apr 09 23:10:14 sistemas1 systemd[1]: Started MySQL Community Server.
root@sistemas1:~#
```


“Cómo podemos ver nuestro MySQL está corriendo de buena forma ahora si podemos continuar”.

Bien ahora como ya sabemos que MySQL ya esta corriendo lo siguiente seria abrir su interfaz y asi poder trabajar en ella.

Para poder realizar eso nosotros debemos ingresar el siguiente comando en la ventana de la terminal del SO de Linux:

```
--mysql -u root -p
```

Con este comando nos debería aparecer los siguiente:

```
root@sistemas1:/home/sistemas1# mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 11
Server version: 8.0.45-0ubuntu0.22.04.1 (Ubuntu)

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```

Luego de esto ingresaremos el comando para poder ver las bases de datos que tenemos en nuestro SO de Linux para eso el siguiente comando:

```
--show databases;
```

```
mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql      |
| performance_schema |
| sys        |
+-----+
4 rows in set (0.01 sec)

mysql> 
```

“Debería aparecernos una lista de las bases de datos que ya tenemos en nuestro servidor local en este caso (root)”.

Una vez estando aquí crearemos una nueva base de datos a la cual denominaremos como sistemas 1 para esto se utiliza el comando siguiente;

```
--create database sistemas1;
```

“Puede ser cualquier nombre luego volvemos a ingresar el *show databases* para verificar que nuestra base de datos se halla creado”.

```
mysql> show databases
-> ;
+-----+
| Database |
+-----+
| information_schema |
| mysql      |
| performance_schema |
| sistemas1   |
| sys        |
+-----+
5 rows in set (0.00 sec)

mysql>
```

Listo ya la tenemos, pero esta base de datos que vamos a usar debería estar vacia asi que vamos a llenarla con unos cuantos datos. Para eso primeramente debemos seleccionar la base de datos que vamos a usar en este caso nuestra base de datos será la de sistemas 1. Usamos el siguiente comando:

```
--use sistemas1;
```

“y para verificar su contenido usamos el comando *--show tables;* que en este caso debería aparecernos vacío”.

```
mysql> use sistemas1
Database changed
mysql> show tables
-> ;
Empty set (0.00 sec)

mysql>
```

Bien ahora nuestro siguiente paso seria crear una tabla dentro de nuestra base de datos y para eso utilizaremos el comando de *--create table nombredelatabla(datos)* justamente como el ejemplo que veremos a continuación:

Cada dato deberá llevar el *nombre tipodedato serdeldato* cada dato que se le va a agregar deber ir separado por una coma, asi como en el ejemplo cada renglón representa que el script no ha terminado, asi que se puede escribir en varios renglones un mismo script.

```
mysql> create table alumnos(id int auto_increment primary key,
-> nota1 double not null,
-> nota2 double not null,
-> nota3 double not null,
-> promedio double not null);
Query OK, 0 rows affected (0.09 sec)
mysql>
```

“Sí nos fijamos en la primera línea que escribimos ahí existe un primary key y un autoincremento esto es porque nuestro id en este caso es la llave primaria y va a tener un auto incremento”.

Luego ingresamos el código `--describe alumnos`; esto porque alumnos le colocamos a nuestra tabla creada. Y con esto nos debería aparecer el contenido de la tabla que acabamos de crear.

```
mysql> describe alumnos
-> ;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id    | int  | NO   | PRI | NULL    | auto_increment |
| nota1 | double | NO   |     | NULL    |               |
| nota2 | double | NO   |     | NULL    |               |
| nota3 | double | NO   |     | NULL    |               |
| promedio | double | NO   |     | NULL    |               |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.02 sec)
mysql>
```

“La tabla nos debería de aparecer de la siguiente manera”.

Bien ahora que lo tenemos debemos de darle ciertos privilegios a nuestra base de datos para eso ingresamos el código `--use mysql`; y luego ingresamos el siguiente código:

```
--create user 'developer'@'%' identified by 'developer2026';
```

“Esto es para crear el usuario al cual le daremos los privilegios, el developer2026 es la contraseña”.

```
--grant all privileges on *.* to 'developer'@'%';
```

```
mysql> create user 'developer'@'%' identified by 'developer2026';
Query OK, 0 rows affected (0.11 sec)

mysql> grant all privileges on *.* to 'developer'@'%' ;
Query OK, 0 rows affected (0.03 sec)

mysql>
```

Y por último ingresamos este código para dejarlo funcional:

```
--flush privileges;
```

Ahora lo que queda es verificar que nuestro usuario halla sido creado para eso ingresamos el código siguiente:

```
--select user, host from user;
```

y nos debería aparecer los usuarios que MySQL trae por defecto y también el usuario que acabamos de crear:

```
mysql> select user, host from user;
+-----+-----+
| user          | host          |
+-----+-----+
| developer     | %             |
| debian-sys-maint | localhost     |
| mysql.infoschema | localhost     |
| mysql.session  | localhost     |
| mysql.sys      | localhost     |
| root          | localhost     |
+-----+-----+
6 rows in set (0.00 sec)

mysql>
```

Listo ya lo tenemos, ahora nuestro siguiente paso será realizar un respaldo de nuestra base de datos asi como también de de la tabla que creamos.

“El % en el host significa que a ese usuario se puedes ingresar desde cualquier lugar mediante la ip”.

Respaldo de nuestra base de datos

para la realización del respaldo de nuestra base de datos tenemos que ingresar el siguiente código, nosotros en este caso mandaremos nuestro respaldo a directorio raíz de nuestro SO de Linux.

```
--mysqldump -u root -p --all-databases --
routines --events --triggers >
/backups_databases.sql
```

“El nombre de backups.databases.sql es el nombre de nuestra base de datos”.

```
root@sistemas1:/home/sistemas1# mysqldump -u root -p --all-databases --routines
--events --triggers > /backups_databases.sql
Enter password:
root@sistemas1:/home/sistemas1#
```

Luego de esto ingresamos a la dirección raíz con un `cd /` y luego enlistaremos el archivo.sql con una `ls -l *.sql`.

```
root@sistemas1:/home/sistemas1# cd /
root@sistemas1:/# ls -l *.sql
-rw-r--r-- 1 root root 1290542 Apr 10 17:28 backups_databases.sql
root@sistemas1:/#
```

Y así es como hemos creado nuestro respaldo de la base de datos que creamos con anterioridad.

Gestionando una partición como respaldo de MySQL

Para la realización del respaldo de una partición para MySQL lo que tenemos que hacer es primeramente realizar una nueva partición en este caso debido al límite de nuestro disco lo haremos de 5 Gb al cual la denominaremos sda4, el procedimiento lo saltaremos ya que en la más arriba se explicó cómo crear una partición lo único que no haremos por el momento será incrustarla dentro del fstab.

```
sda      8:0    0    51G  0 disk
├─sda1   8:1    0    1M  0 part
├─sda2   8:2    0    25G  0 part /
├─sda3   8:3    0    10G  0 part /home/username/Desktop/mnt/sda3
└─sda4   8:4    0    5G   0 part
```

Listo ya la hemos creado. Ahora lo que sigue es dentro de la carpeta mnt de la dirección raíz hay que crear un directorio DataDB y dentro de este directorio montaremos nuestra partición sda4 “sin incrustarla en el fstab por ahora”, recuerda que antes de montar la partición hay que darle el formato ext4, sino dará errores.

Ahora que lo tenemos creado, lo siguiente será montar la partición sda4 solamente con

el comando `sudo mount` justamente en el directorio DataDB.

```
root@sistemas1:/mnt# mkdir DataDB
root@sistemas1:/mnt# ls -l
total 4
drwxr-xr-x 2 root root 4096 Apr 12 21:22 DataDB
root@sistemas1:/mnt# sudo mount /dev/sda4 /mnt/DataDB
root@sistemas1:/mnt# cd DataDB
```

“Y aquí tenemos hecho el montaje de la partición sda4 en el directorio DataDB”

Nuestro siguiente paso deberá ser crear un directorio dentro de DataDB al cual lo denominaremos como `mysql_s1`.

```
drwx----- 2 root root 16384 Apr 11 15:14 lost+found
drwxr-xr-x 2 root root 4096 Apr 11 15:59 mysql_s1
```

Bien ahora que lo tenemos listo nuestro primer proceso es ir a ubicarnos en el directorio de nuestra partición sda4 que creamos anteriormente.

```
root@sistemas1:/mnt/DataDB#
```

Bien ahora que lo tenemos vamos a darle permisos a la partición sda4 que acabamos de crear para hacerla de tipo MySQL para eso utilizamos los siguientes comandos:

```
--sudo chown -R mysql:mysql
direcciónPartición
```

```
root@sistemas1:/mnt/DataDB# sudo chown -R mysql:mysql /mnt/DataDB/mysql_s1
```

Si nos fijamos ya nuestros directorios ya están convertidos a tipo MySQL ya que anteriormente aparecía de tipo root ahora ya nos aparece de tipo MySQL.

```
drwx----- 2 mysql mysql 16384 Apr 11 15:14 lost+found
drwxr-xr-x 2 mysql mysql 4096 Apr 11 15:59 mysql_s1
```

“Y en efecto nuestro directorio creado si es de tipo MySQL”

Ahora si vamos a darle los permisos, el comando es muy parecido al que utilizamos en los tipos de archivos.sh y archivos.html solo que con un pequeño cambio:

```
--sudo chmod 750 direccionmysql_s1
```

```
root@sistemas1:/mnt/DataDB# sudo chmod 750 /mnt/DataDB/mysql_s1
```

Listo nuestro directorio de `mysql_s1` debería aparecer en su inicio lo siguiente:

```
root@sistemas1:/mnt/DataDB# ls -l
total 20
drwx----- 2 mysql mysql 16384 Apr 11 15:14 lost+found
drwxr-x--- 2 mysql mysql 4096 Apr 12 21:17 mysql_s1
```

--Drwxr-x---

Una vez listo lo siguiente antes de continuar es verificar si nuestro MySQL server está corriendo, aplicando el siguiente comando:

--/etc/init.d/mysql status

```
root@sistemas1:~# /etc/init.d/mysql status
mysql.service - MySQL Community Server
Loaded: loaded (/lib/systemd/system/mysql.service; enabled; vendor preset: enabled)
Active: active (running) since Thu 2020-04-09 23:10:14 UTC; 22min ago
Process: 860 ExecStartPre=/usr/share/mysql/mysql-systemd-start pre (code=exited, status=0/SUCCESS)
Main PID: 908 (mysqld)
Status: "Server is operational"
Tasks: 37 (limit: 4547)
Memory: 423.2M
CPU: 26.228s
CGroup: /system.slice/mysql.service
        └─ 908 /usr/sbin/mysqld

Apr 09 23:10:05 sistemas1 systemd[1]: Starting MySQL Community Server...
Apr 09 23:10:14 sistemas1 systemd[1]: Started MySQL Community Server.
root@sistemas1:~#
```

Nuestro MySQL está corriendo de buena forma ahora, pero para los siguientes pasos necesitamos apagarlo, para apagarlo utilizaremos el siguiente comando:

--/etc/init.d/mysql stop

Y una vez lo tengamos, tenemos que verificar que el estatus del MySQL este totalmente apagado.

--/etc/init.d/mysql status

```
root@sistemas1:/home/username/Desktop/mnt/DataDB# /etc/init.d/mysql stop
Stopping mysql (via systemctl): mysql.service.
root@sistemas1:/home/username/Desktop/mnt/DataDB# /etc/init.d/mysql status
mysql.service - MySQL Community Server
Loaded: loaded (/lib/systemd/system/mysql.service; enabled; vendor preset: enabled)
Active: inactive (dead) since Sat 2020-04-11 16:36:44 UTC; 3s ago
Process: 860 ExecStartPre=/usr/share/mysql/mysql-systemd-start pre (code=exited, status=0/SUCCESS)
Main PID: 922 ExecStart=/usr/sbin/mysqld (code=exited, status=0/SUCCESS)
Status: "Server shutdown complete"
CPU: 1min 36.821s

Apr 11 14:55:34 sistemas1 systemd[1]: Starting MySQL Community Server...
Apr 11 14:55:49 sistemas1 systemd[1]: Started MySQL Community Server.
Apr 11 16:36:42 sistemas1 systemd[1]: Stopping MySQL Community Server...
Apr 11 16:36:44 sistemas1 systemd[1]: mysql.service: Deactivated successfully.
Apr 11 16:36:44 sistemas1 systemd[1]: Stopped MySQL Community Server.
Apr 11 16:36:44 sistemas1 systemd[1]: mysql.service: Consumed 1min 36.821s CPU time.
```

Ingreso de nuestra partición a la datadir de mysql

Listo, lo siguiente es irnos a nuestra dirección raíz, para seguir en lo siguiente es de vital importancia que se halla montado la partición en la raíz del SO de Linx. Lo siguiente que hay que hacer es dirigirnos a la siguiente ruta de acceso:

--cd /etc/mysql/mysql.cnf.d

Dentro de esta dirección encontraremos los siguientes directorios y archivos almacenados:

```
root@sistemas1:/etc/mysql/mysql.conf.d# ls -l
total 8
-rw-r--r-- 1 root root 132 Nov 18 20:53 mysql.cnf
-rw-r--r-- 1 root root 2220 Nov 18 20:53 mysqld.cnf
root@sistemas1:/etc/mysql/mysql.conf.d#
```

A nosotros el que nos interesa el archivo `mysqld.cnf`. ingresaremos al archivo con pico:

--pico mysqld.cnf

Dentro del archivo nos aparecerá lo siguiente:

```
GNU nano 6.2                                mysqld.cnf
#
# The MySQL database server configuration file.
#
# One can use all long options that the program supports.
# Run program with --help to get a list of available options and with
# --print-defaults to see which it would actually understand and use.
#
# For explanations see
# http://dev.mysql.com/doc/mysql/en/server-system-variables.html
#
# Here is entries for some specific programs
# The following values assume you have at least 32M ram

[mysqld]
#
# * Basic Settings
#
user                    = mysql
# pid-file              = /var/run/mysqld/mysqld.pid
# socket                = /var/run/mysqld/mysqld.sock
# Read 78 lines
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File ^_ Replace   ^U Paste     ^J Justify
```

Debajo de lo ultimo, en este caso de donde termina los comentarios de `user` y `mysql` agregaremos lo siguiente:

--datadir = direccióndelmysql_s1

```
GNU nano 6.2                                mysqld.cnf
#
# The MySQL database server configuration file.
#
# One can use all long options that the program supports.
# Run program with --help to get a list of available options and with
# --print-defaults to see which it would actually understand and use.
#
# For explanations see
# http://dev.mysql.com/doc/mysql/en/server-system-variables.html
#
# Here is entries for some specific programs
# The following values assume you have at least 32M ram

[mysqld]
#
# * Basic Settings
#
user                    = mysql
# pid-file              = /var/run/mysqld/mysqld.pid
# socket                = /var/run/mysqld/mysqld.sock
# port                  = 3306
# datadir               = /var/lib/mysql
datadir                 = /mnt/DataDB/mysql_s1
# If MySQL is running as a replication slave, this should be
```

Una vez lo tengamos, lo que tenemos que hacer el guardar el archivo (`ctrl+o`), y después salirnos del mismo (`ctrl+x`).

Bien ahora que tenemos este archivo editado y guardado lo siguiente es dirigirnos

nuevamente a otra ubicación la cual será la siguiente:

```
--cd /etc/apparmor.d
```

Y una vez estemos dentro de esta dirección lo que tenemos que hacer es editar un archivo que se encuentra dentro de esta dirección en este caso el archivo *usr.bin.mysql*.

```
root@sistemas1:/etc/apparmor.d# ls -l
total 156
drwxr-xr-x 2 root root 4096 Apr  8 06:08 abi
drwxr-xr-x 4 root root 12288 Apr  8 06:08 abstractions
drwxr-xr-x 2 root root 4096 Sep 11 2024 disable
drwxr-xr-x 2 root root 4096 Apr  8 05:49 force-complain
drwxr-xr-x 2 root root 4096 Apr  9 23:00 local
-rw-r--r-- 1 root root 1339 Aug 27 2024 lsb_release
-rw-r--r-- 1 root root 1190 Aug 15 2025 nvidia_nodprobe
-rw-r--r-- 1 root root 3500 Jan 31 2023 sbin.dhclient
drwxr-xr-x 5 root root 4096 Apr  8 06:08 tunables
-rw-r--r-- 1 root root 2864 Oct 27 12:42 ubuntu_pro_spt_news
-rw-r--r-- 1 root root 7050 Oct 27 12:42 ubuntu_pro_esn_cache
-rw-r--r-- 1 root root 11495 Oct 19 2023 usr.bin.evince
-rw-r--r-- 1 root root 3448 Mar 17 2022 usr.bin.man
-rw-r--r-- 1 root root 1687 Feb  8 2024 usr.bin.tcpdump
-rw-r--r-- 1 root root 1519 May  5 2025 usr.lib.libreoffice.program.oosplash
-rw-r--r-- 1 root root 1227 May  5 2025 usr.lib.libreoffice.program.senddoc
-rw-r--r-- 1 root root 18740 May  5 2025 usr.lib.libreoffice.program soffice.bin
-rw-r--r-- 1 root root 1046 May  5 2025 usr.lib.libreoffice.program.xpdfimport
-rw-r--r-- 1 root root 32729 Mar 12 11:30 usr.lib.snapd.snap-confine.real
-rw-r--r-- 1 root root  677 Apr  6 2022 usr.sbin.cups-browsed
-rw-r--r-- 1 root root 6141 Dec  3 14:44 usr.sbin.cupsd
-rw-r--r-- 1 root root 2086 Jan 22 12:44 usr.sbin.mysql
-rw-r--r-- 1 root root 1592 Nov 16 2021 usr.sbin.rsyslogd
-rw-r--r-- 1 root root 1668 Aug 12 2025 usr.sbin.sssd
root@sistemas1:/etc/apparmor.d#
```

```
--pico usr.sbin.mysql
```

Y entrando en el archivo nos debería aparecer lo siguiente:

```
GNU nano 6.2
# vim:syntax=apparmor
# Last Modified: Tue Feb 09 15:28:30 2016
#include <tunables/global>

/usr/sbin/mysql {
    #include <abstractions/base>
    #include <abstractions/nameservice>
    #include <abstractions/user-tmp>
    #include <abstractions/mysql>
    #include <abstractions/winbind>

# Allow system resource access
/proc/* /status r,
/sys/devices/system/cpu/ r,
/sys/devices/system/node/ r,
/sys/devices/system/node/** r,
capability sys_resource,
capability dac_override,
capability dac_read_search,
capability setuid,
capability setgid,

# Allow network access
network tcp,
```

Justamente teniendo esto, nos vamos a mover hasta la última línea de este archivos justo antes del corchete que aparece de ultimo, en esa zona nosotros debemos colocar la dirección de nuestra partición DataDB para darle permisos de escritura, lectura y una de

tipo LOCK. Esto tanto a los directorios como también a los subdirectorios.

```
GNU nano 6.2                                     usr.sbi
# Allow data files dir access
/var/lib/mysql-files/ r,
/var/lib/mysql-files/** rwk,

# Allow keyring dir access
/var/lib/mysql-keyring/ r,
/var/lib/mysql-keyring/** rwk,

# Allow log file access
/var/log/mysql.err rw,
/var/log/mysql.log rw,
/var/log/mysql/ r,
/var/log/mysql/** rw,

# Allow read access to mecab files
/var/lib/mecab/dic/tpadic-utf8/** r,

# Allow read access to OpenSSL config
/etc/ssl/openssl.cnf r,
# Site-specific additions and overrides. See local/README for details.
#include <local/usr.sbin.mysql>

/mnt/DataDB/mysql_s1/ r,
/mnt/DataDB/mysql_s1/** rwk,
}
```

Debemos colocarlas como las últimas dos líneas que aparecen antes del corchete. Lo que colocamos en si, es la dirección de nuestra partición DataDB. Listo, lo guardamos y cerramos.

Una vez cerrado el archivo, lo siguiente es darle permisos al directorio de mysql_1 esto con el siguiente código:

```
--sudo chown -R mysql:mysql
mnt/DataDB/mysql_s1
```

“Recordar que la dirección de todo debe estar de tipo mysql sino al momento de iniciar el MySQL nos dará error.” Deberíamos ingresar los siguientes comandos asegurándonos de que todo sea de tipo mysql.

```
root@sistemas1:/# sudo chown -R mysql:mysql mnt
root@sistemas1:/# sudo chown -R mysql:mysql mnt/DataDB
root@sistemas1:/# ls -l mnt
total 4
drwxr-xr-x 4 mysql:mysql 4096 Apr 12 22:13 DataDB
root@sistemas1:/#
```

Para inicializar la datadir de la particion que creamos, tenemos que ingresar el siguiente código:

```
--sudo mysql --initialize-insecure --
user=mysql
datadir=/mnt/DataDB/mysql_s1
```

“La dirección va a depender de la ruta que hallamos puesto en el primer archivo que editamos.”


```
root@sistemas1:/mnt/Data08# sudo mysqld initialize --user=mysql --datadir=/mnt/Data08/mysql_x1
root@sistemas1:/mnt/Data08#
```

Si todo sale bien no debería haber ningún error con el código que ingresamos.

Y es así como hemos creado un backup de nuestro MySQL en una partición aparte.

Automatización del proceso de backups MySQL

Bien ahora que ya hemos visto todo el proceso que se hace para la realización de Backups para así tener un respaldo de nuestra base de datos de MySQL ahora lo que tenemos es automatizar todo esto, para esto haremos la utilización de los archivos.sh y también (muy importante) la base de datos que hicimos con anterioridad la cual la denominamos como "sistemas1".

Nuestro primer paso será asegurarnos de que nuestro MySQL este corriendo con el comando:

`--/etc/init.d/mysql status`

```
root@sistemas1:~# /etc/init.d/mysql status
mysql.service - MySQL Community Server
Loaded: loaded (/lib/systemd/system/mysql.service; enabled; vendor preset: enabled)
Active: active (running) since Thu 2026-04-09 23:10:14 UTC; 22min ago
Process: 900 ExecStartPre=/usr/share/mysql/mysql-systemd-start pre (code=exited, status=0/SUCCESS)
Main PID: 908 (mysqld)
Status: "Server is operational"
Tasks: 37 (limit: 4547)
Memory: 423.2M
CPU: 29.219s
CGroup: /system.slice/mysql.service
└─908 /usr/sbin/mysqld

Apr 09 23:10:05 sistemas1 systemd[1]: Starting MySQL Community Server...
Apr 09 23:10:14 sistemas1 systemd[1]: Started MySQL Community Server.
root@sistemas1:~#
```

Una vez lo tengamos abriremos nuestro MySQL con el comando `--mysql -u root -p` ingresaremos a la base de datos sistemas1 y crearemos un usuario igual al anterior, le pondremos developer y su contraseña como developer 2026.

```
root@sistemas1:~# mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 11
Server version: 8.0.45-0ubuntu22.04.1 (Ubuntu)

Copyright (c) 2000, 2026, Oracle and/or its affiliates.
Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use sistemas1;
```

"simplemente que el host lo dejaremos como localhost y no como %"

```
mysql> create user 'developer'@'localhost' identified by 'developer2026';
Query OK, 0 rows affected (0.06 sec)
```

Y por último le damos full privilegios a nuestro usuario con el comando:

```
--grant all privileges on *.* to
'developer'@"localhost";
```

```
mysql> grant all privileges on *.* to 'developer'@'localhost';
Query OK, 0 rows affected (0.03 sec)

mysql>
```

Una vez lo tengamos vamos a irnos a la raíz de nuestro SO (hay que salirse del MySQL). con un `cd` y aquí vamos a crear nuestro archivo.sh siempre con el comando `pico`:

Una vez lo tengamos, vamos a irnos a la raíz de nuestro SO con un `cd`, y aquí vamos a crear nuestro archivo.sh siempre con el comando `pico`:

`--pico BackupData.sh`

```
GNU nano 6.2 BackupData.sh *
#!/bin/bash
```

"Importante colocar el `#!/bin/bash` para que el archivo.sh funcione correctamente".

Una vez lo tenemos vamos a crear unas variables que van a almacenar nuestros nombres y direcciones.

```
#!/bin/bash
USER="developer"
PASS="developer2026"
HOST="localhost"
DATABASE="sistemas1"
PATH_DB="/home/username/Desktop/mnt/sda3/DataBK"
DATE=$(date +%Y-%m-%d_%H-%M-%S)
```

Lo primero es agregar las variables el usuario que creamos como developer, la contraseña que le pusimos que era developer2026, el host que en este caso es "%", la base de datos que era sistemas1, el path que es donde vamos a guardar nuestro backup y por último la fecha que se la vamos a añadir al nombre del archivo.

```
#crear carpeta
mkdir -p $PATH_DB
BACKUP_FILE="$PATH_DB/$DATABASE-$DATE.sql"

#realizar el backup
mysqldump -u $USER -p$PASS -h $HOST $DATABASE > $BACKUP_FILE
```


Una vez lo tengamos listo nuestro siguiente paso será hacer la creación de la carpeta y la realización del backup el cual lo realizaremos con el comando mysqldump.

```
if [ $? -eq 0 ]
then
    echo "Respaldo hecho exitosamente"
else
    echo "Su Respaldo ha sido erroneo"
fi
```

Y por último agregaremos estos if y else para asegurarnos de que nuestro archivo.sh salga de mil maravillas.

Antes de probar el programa, debemos darle permisos de edición y ejecución con el siguiente comando:

```
--sudo chmod -R 777 BackupData.sh
```

```
root@sistemas1:/# sudo chmod -R 777 BackupData.sh
```

Ahora vamos a probar el archivo.sh con el comando siguiente `--./BackupData.sh`.

```
root@sistemas1:/# ./BackupData.sh
mysqldump: [Warning] Using a password on the command line interface can be insecure.
Respaldo hecho exitosamente
```

Como podemos ver nuestro respaldo ha sido creado exitosamente. Ahora vamos a comprobarlo hciendo a la dirección que le pusimos al path a ver si se creó nuestro directorio.

```
root@sistemas1:/home/username/Desktop/mnt/sda3# ls -l
total 28
drwxr-xr-x 2 root root 4096 Apr 14 00:13 DataBK
drwxrwxrwx 2 root root 16384 Apr 8 22:25 lost+found
drwxrwxrwx 2 mysql mysql 4096 Apr 9 23:16 mysql
drwxrwxrwx 2 root root 4096 Apr 9 19:12 public_app
```

Y como podemos ver, nuestro directorio se logró crear sin ningún problema. Si ingresamos el comando `ls -l DataBK` se nos enlistará los respaldos que se hicieron con la fecha que se creó.

```
root@sistemas1:/home/username/Desktop/mnt/sda3# ls -l DataBK
total 8
-rw-r--r-- 1 root root 1961 Apr 14 00:13 sistemas1-2026-04-14_00-13-50.sql
-rw-r--r-- 1 root root 1961 Apr 14 00:17 sistemas1-2026-04-14_00-17-17.sql
```

Bien ahora lo que sigue es hacer que este archivo se ejecute de manera automática todos los días, para eso nos vamos ir directamente a la raíz de Linux y creamos un

archivo con el comando *pico* al cual lo llamaremos como actividad1.

```
--pico actividad1
```

```
root@sistemas1:/# pico actividad1
```

Y pondremos lo siguiente, cada “*” significa: minutos, hora, mes, día, día(entre semana), y el root es el que se va a encargar de ejecutar el BackupData.sh.

Nosotros lo dejaremos de la siguiente manera debido a nuestra hora, para que podamos ver que si se crea el archivo de BackupData.

Apr 14 20:15

“En mi caso la hora que tengo es la 8 con 15 de la noche, estableceré que el programa se ejecute a las 8 y 17, para tener un margen de error y evitar que no se cree el archivo”.

```
GNU nano 6.2 actividad1 *
17 20 * * * root /BackupData.sh
```

Estableciendo esto dejando las 3 almohadillas establezco que se haga todos los días durante todos los meses.

Luego de guardar el archivo debemos ingresar el comando para darle permisos de edición al archivo y que se pueda ejecutar:

```
--sudo chmod -R 755 actividad1
```

```
root@sistemas1:/# sudo chmod 755 actividad1
root@sistemas1:/# /etc/init.d/cron restart
Restarting cron (via systemctl): cron.service.
root@sistemas1:/# date
Tue 14 Apr 20:17:00 CST 2026
```

Luego reinicias el cron con el comando `/etc/init.d/cron restart` una vez se ha reiniciado, revisamos la fecha y ya son las 8 y 17 nos dirigimos a la ruta del archivo.sh y estando ahí ejecutamos el comando `tree` y ya debería aparecer el backup que hicimos.

```
root@sistemas1:/# cd /home/username/Desktop/mnt/sda3
root@sistemas1:/home/username/Desktop/mnt/sda3# tree
.
├── Data8K
│   ├── sistemas1-2026-04-13_19-39-48.sql
│   ├── sistemas1-2026-04-14_00-13-50.sql
│   ├── sistemas1-2026-04-14_00-17-17.sql
│   └── sistemas1-2026-04-14_20-17-32.sql
├── lost+found
├── mysql_s1
├── public_app1
└── index.html

4 directories, 5 files
```

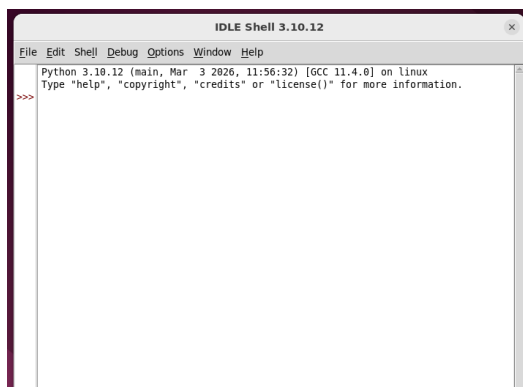
Y nuestro archivo se creó correctamente, aquí se establece la creación del archivo con la hora y fecha que le establecimos.

Utilizando la interfaz de Python en Linux

Bien ahora que hemos terminado con MySQL lo siguiente será pasar a utilizarlo lo hecho de otra manera, para eso vamos a utilizar Python para eso lo tenemos que instalar en nuestra maquina, para eso tenemos el siguiente comando:

```
--sudo apt-get install python3 python3-tk idle
```

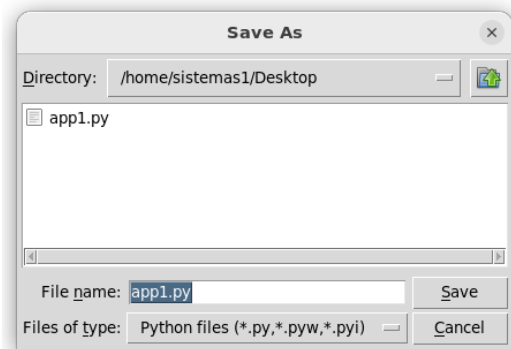
Una vez instalado nuestro siguiente paso será buscar el iniciador de python idle en nuestros programas, pero antes de esto debemos reiniciar nuestro sistemas operativo para evitar errores, deberá aparecernos de la siguiente manera:



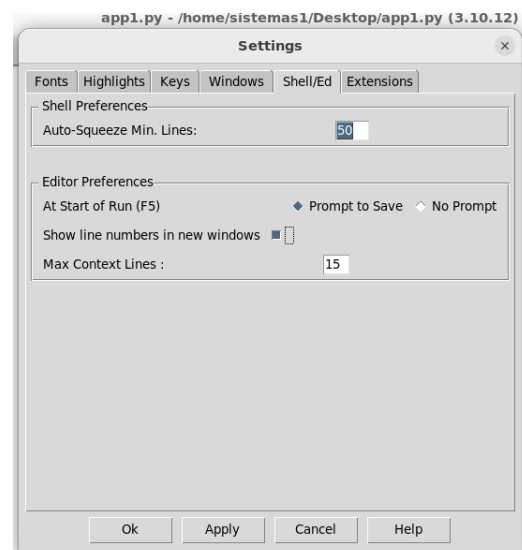
Luego entramos al idle de nuestro python (el programa) y nos debería aparecer esto al entrar nosotros vamos a darle ctrl+n para crear un nuevo archivo y así poder empezar a programar lo que necesitamos.



Nos debería aparecer esto jaja. Ahora bien, lo siguiente será empezar a programar, pero antes hay que guardar el archivo.py. en mi caso lo guardare en el directorio de escritorio de sistemas1.



Luego de eso vamos irnos a settings y vamos a habilitar el número de línea:



Primero Importamos las siguientes librerías en nuestro archivo.py:

```

1 #Developer: Universidad
2 from tkinter import *
3 from tkinter import ttk
4 from tkinter import font
5 from tkinter import messagebox
6 import subprocess
7 from tkinter import scrolledtext
8 import os
9 import time

```

Una vez lo tengamos vamos a crear un código para crear una ventana

```

11 #crear una ventana
12 v0=Tk()
13 v0.title("Herramientas del sistemas de linux")
14 v0.geometry("800x600+200+200")
15
16 v0.mainloop()
17
18

```

Y teniéndolo aquí lo que nos debería aparecer al correr el programa seria lo siguiente:



Lo vamos a tener que hacer un poco mas pequeño porque está demasiado grande, pero eso lo haremos después, lo siguiente seria ingresar las fuentes, las etiquetas y la zona de los botones con el siguiente código.

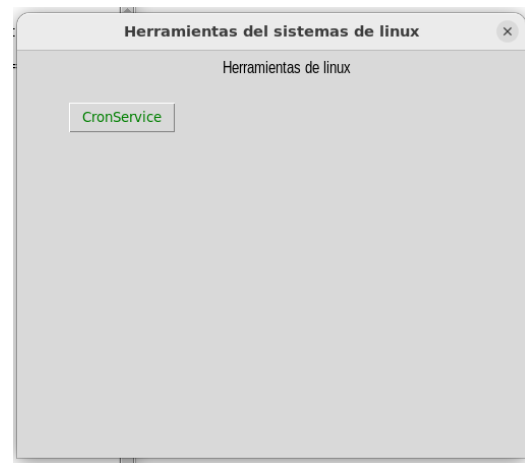
```

11 #Zona de funciones
12 def CronService1():
13     print("Ejecutando CronService1")
14
15 #crear una ventana
16 v0=Tk()
17 v0.title("Herramientas del sistemas de linux")
18 v0.geometry("500x400+200+200")
19
20 #fuentes
21 text1=font.Font(family="Arial",size=25)
22 text2=font.Font(family="Arial Narrow",size=12)
23
24 #Zona de etiquetas
25 label_t=Label(v0,text="Herramientas de linux",font=text1).place(x=200,y=5)
26
27 #zona de botones
28 btn_cron1=Button(v0,text="CronService1",fg="green",command=CronService1).place(x=50,y=50)
29
30 v0.mainloop()
31

```

Este seria nuestro código estableciendo la zona de las funciones, la creación de la ventana, las fuentes, las etiquetas y los botones una ve lo tenemos corremos el

programa, y nos debería aparecer lo siguiente:



Estando aquí al momento de hacer click en el botón que creamos debería aparecernos lo siguiente en pantalla:

```

Python 3.10.12 (main, Mar 3 2026, 11:56:32) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: /home/sistemas1/Desktop/app1.py =====
Ejecutando CronService1
Ejecutando CronService1

```

Nuestro siguiente objetivo será crear esta función a la cual llamaremos como CrearBK para hacer este paso necesitaremos haber creado el archivo.sh del backup que realizamos dentro del sda3, prácticamente necesitamos su dirección del para eso usamos el código

```
--os.system("pkexec /BackupData")
```

Lo normal es usar `--os.system("sudo /BackupData &")` pero esto nos dará errores y no se generara el archivo asi por eso usamos el código de arriba

Y luego usaremos el código de:

```
--
messagebox.showinfo("INFO",message="Backup realizado exitosamente")
```

Esto para asegurarnos de que nuestro backup se halla realizado de forma correcta. El código debería quedarnos asi:

```

11 #Zona de funciones
12 def CronService1():
13     print("Ejecutando CronService1")
14 def Crear_BK():
15     os.system("pexec /BackupData.sh")
16     messagebox.showinfo("INFO",message="Backup Realizado con Exito")
17

```

Y luego crearemos el Botón para poder ejecutar la función agregándolo a la zona de los botones, más o menos debería quedarnos así:

```

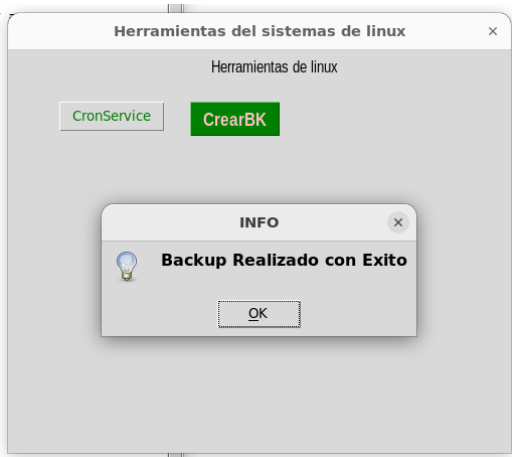
11 #Zona de funciones
12 def CronService1():
13     print("Ejecutando CronService1")
14 def Crear_BK():
15     os.system("pexec /BackupData.sh")
16     messagebox.showinfo("INFO",message="Backup Realizado con Exito")
17
18 #crear una ventana
19 vb=Frame()
20 vb.title("Herramientas del sistemas de linux")
21 vb.geometry("500x400+200+200")
22
23 #fuentes
24 text=Font(family="Arial",size=20)
25 text1=Font(family="Arial Narrow",size=12)
26
27 #zona de etiquetas
28 label1=Label(vb,text="Herramientas de linux",font=text1,place(x=200,y=50))
29
30 #zona de botones
31 btn_cron=Button(vb,text="CronService",fg="green",command=CronService1,place(x=50,y=50))
32 btn_bk=Button(vb,text="CrearBK",bg="green",fg="pink",font=("Arial Narrow",14,"bold"),command=Crear_BK,place(x=180,y=50))
33

```

Una vez tengamos el script así, nos debería aparecer la ventana de la siguiente forma:



Ahora probaremos el botón a ver que sucede. Cuando lo usemos siempre nos pedirá la contraseña.



Al parecer todo va bien, para ver si esto es real vamos a irnos a la ruta donde se guardan los backups en el sda3, vamos allá:

```

root@sisemas1:/home/username/Desktop/mnt/sda3# tree
.
├── DataBK
│   ├── sistemas1-2026-04-13_19-39-48.sql
│   ├── sistemas1-2026-04-14_00-13-50.sql
│   ├── sistemas1-2026-04-14_00-17-17.sql
│   ├── sistemas1-2026-04-14_20-17-32.sql
│   ├── sistemas1-2026-04-15_21-36-31.sql
│   ├── sistemas1-2026-04-15_21-41-30.sql
│   └── sistemas1-2026-04-15_21-42-47.sql
├── lost+found
├── mysql s1
├── public app1
│   └── index.html
└── Python

```

Y si nos fijamos el archivo y el botón funcionan de buena forma.

Ahora lo que vamos a hacer es que todos estos archivos generan una lista para luego luego ingresar los registros en un archivo de tipo txt.

Lo primero es crear el archivo.txt para eso nos vamos a ir a la ruta donde se guardan los backups y dentro de ella vamos a crear el archivo.txt y le vamos a dar permisos de lectura y escritura --sudo chmod -R 777 listadobk.txt y nos debería quedar así:

```

root@sisemas1:/home/username/Desktop/mnt/sda3# cd DataBK
root@sisemas1:/home/username/Desktop/mnt/sda3/DataBK# sudo chmod -R 777 listadobk.txt
root@sisemas1:/home/username/Desktop/mnt/sda3/DataBK# ls -l
total 28
-rwxrwxrwx 1 root root  0 Apr 15 21:56 listadobk.txt
-rw-r--r-- 1 root root 1961 Apr 13 19:39 sistemas1-2026-04-13_19-39-48.sql
-rw-r--r-- 1 root root 1961 Apr 13 18:13 sistemas1-2026-04-14_00-13-50.sql
-rw-r--r-- 1 root root 1961 Apr 13 18:17 sistemas1-2026-04-14_00-17-17.sql
-rw-r--r-- 1 root root 1961 Apr 14 20:17 sistemas1-2026-04-14_20-17-32.sql
-rw-r--r-- 1 root root 1961 Apr 15 21:36 sistemas1-2026-04-15_21-36-31.sql
-rw-r--r-- 1 root root 1961 Apr 15 21:41 sistemas1-2026-04-15_21-41-30.sql
-rw-r--r-- 1 root root 1961 Apr 15 21:42 sistemas1-2026-04-15_21-42-47.sql
root@sisemas1:/home/username/Desktop/mnt/sda3/DataBK#

```

Ahora que lo tenemos vamos a regresar al script de python y vamos a agregar una función y un botón para agregar la lista en nuestra ventana, empezaremos con esta función, primeramente:

```

17 def Listar_BK():
18     os.system("ls -l /home/username/Desktop/mnt/sda3/DataBK > /home/username/Desktop/mnt/sda3/DataBK/listadobk.txt")
19     valor=subprocess.check_output("cat /home/username/Desktop/mnt/sda3/DataBK/listadobk.txt",shell=True)
20     area1=ScrolledText.ScrolledText(vb,width=50,height=10)
21     area1.place(x=50,y=180)
22     area1.insert(END,valor)

```

Utilizaremos el os.systema para enlistar los backups que se crean dentro del directorio de DataBK para luego mandarlos al archivo.txt y poderlos mostrar en pantalla.

```

--os.system("ls -l /home/username/Desktop/mnt/sda3/DataBK

```

>

```
/home/username/Desktop/mnt/sda3/DataBK  
/listadobk.txt")
```

Luego con el subprocess enlistaremos los archivos de los backups y así poderlos mostrar dentro del cuadro que vamos a crear.

```
--valor1=subprocess.check_output("cat  
/home/username/Desktop/mnt/sda3/DataBK  
/listadobk.txt",shell=True)
```

Listo eso, estableceremos el área del recuadro que nos va a aparecer así como también en la ubicación del mismo:

--

```
area1=scrolledtext.ScrolledText(v0,width=50  
,height=10)
```

```
--area1.place(x=50,y=110)
```

```
--area1.insert(END,valor1)
```

Quedándonos así:

```
17 def Listar_BK():  
18     os.system("ls -l /home/username/Desktop/mnt/sda3/DataBK > /home/username/Desktop/mnt/sda3/DataBK/listadobk.txt")  
19     valor1=subprocess.check_output("cat /home/username/Desktop/mnt/sda3/DataBK/listadobk.txt",shell=True)  
20     area1=scrolledtext.ScrolledText(v0,width=50,height=10)  
21     area1.place(x=50,y=110)  
22     area1.insert(END,valor1)
```

Una vez lo tenemos, lo último sería ingresar el botón que va a hacer la magia y para eso lo ingresamos en la zona de los otros botones dándole estas características:

```
#zona de botones  
btn_cron=Button(v0,text="CronService",fg="green",command=CronService).place(x=50,y=50)  
btn_creabk=Button(v0,text="CrearBK",fg="green",fg="pink",font=("Arial Narrow",14,"bold"),command=Crear_BK).place(x=300,y=50)  
btn_listarbk=Button(v0,text="ListarBK",fg="green",fg="pink",font=("Arial Narrow",14,"bold"),command=Listar_BK).place(x=310,y=50)
```

--

```
btn_bk=Button(v0,text="ListarBK",bg="green  
",fg="pink",font=("Arial  
Narrow",14,"bold"),command=Listar_BK).pla  
ce(x=310,y=50)
```

Ahora si llegó el momento de la verdad, vamos a correr el programa, y nos debería aparecer así:



Y ahora le damos click en el botón de listarbk y ya nos debería aparecer la lista de los backups que pasamos al archivo.txt anterior:



Y es así como creamos este pequeño programa que realiza los backups de forma automatizada y también los enlista. Quedó mágica la cosa.